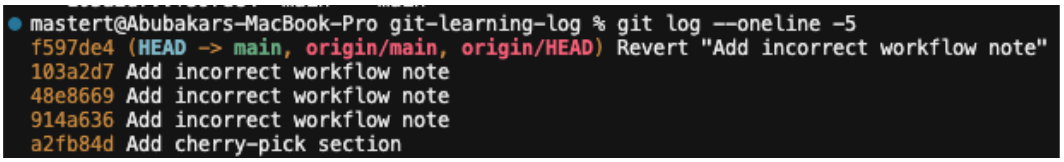
GIT UNDO, RECOVERY, AND DEBUGGING

July 2026



Project Overview: Mastering Git Recovery and Debugging

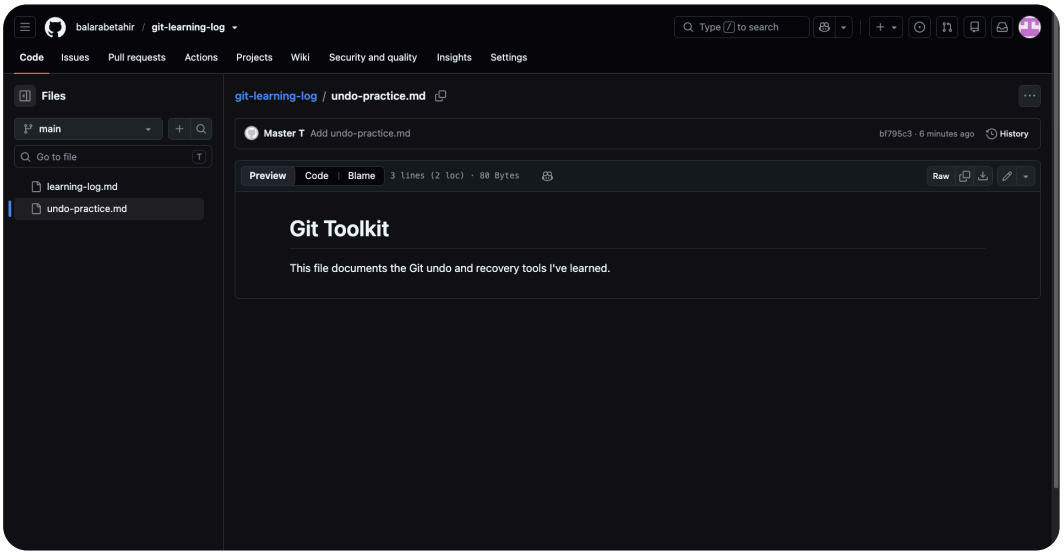
What this project set out to achieve

In this project, I'm learning how to undo mistakes, recover lost commits, and debug history using Git's reset, revert, reflog, cherry-pick, and bisect tools. I'm building a practice repository to safely test these recovery techniques without affecting real code. I'm also learning to tag releases for CI/CD integration. So that I can confidently fix broken code, restore accidentally deleted work, and track down bugs efficiently in any collaborative or solo project without fear of permanent data loss.

Setting Up the Repository and Environment

Goals for this setup step

In this step, I'm setting up my complete development environment by installing Cursor as my code editor, verifying Git is installed correctly, and making sure my GitHub account is connected. I am then cloning or opening the git-learning-log repository in Cursor. Finally, I am creating a starting file and pushing it to GitHub. So that I can have a clean, stable foundation for all the undo and recovery experiments that follow, ensuring every practice session works correctly from the very beginning without any setup issues.



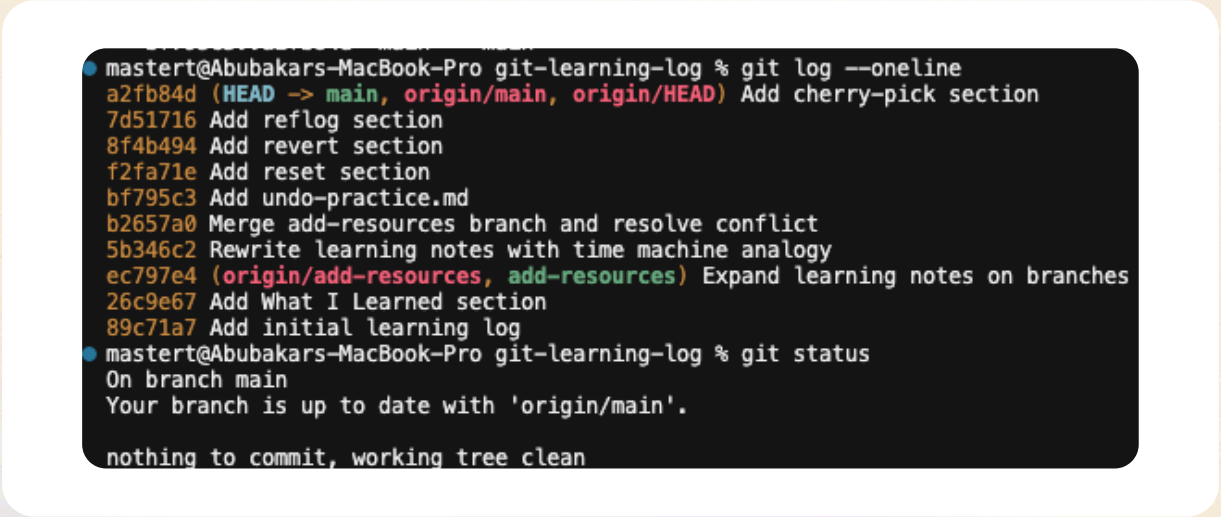
Configuring Git for this project

I configured my user name, email address, and default editor using git config with the --global flag. I set my name and email so every commit I make is properly attributed to me. I set the editor to nano because it is simpler than vim and makes writing commit messages much less confusing for beginners. This configuration applies to all my repositories, which saves time and prevents errors. I did this so that my commits are correctly identified and I can comfortably write messages when commands like git revert open an editor.

Building a Meaningful Commit History

Why a rich history matters for practice

In this step, I'm building a commit history by adding four documentation sections to my undo-practice.md file, each saved as a separate commit. I am writing about reset, revert, reflog, and bisect commands. I am staging and committing each section one by one with clear messages. Finally, I am pushing all commits to GitHub and verifying my history looks clean. So that I can have a realistic practice ground with multiple commits to experiment on, allowing me to safely test undo and recovery commands without affecting any important real project.



The value of granular commits

I committed separately because each section represents a distinct topic that I want to practice on individually. Git's undo and recovery tools work on specific commits, so having multiple small commits gives me more realistic targets to experiment with. I can reset to one commit, revert another, or cherry-pick a specific change. This makes my practice much more effective. If I had combined everything into one big commit, I would not have the flexibility to test these commands properly. So I built a detailed history that mimics real project work.

Undoing Mistakes Safely with Reset and Revert

Approach to undoing bad commits

In this step, I'm learning how to undo a local commit using git reset --soft and how to safely undo a pushed commit using git revert. I am practicing resetting my last commit while keeping changes staged, then cleaning up my working directory. I am also creating a revert commit that undoes a previous change without rewriting history. I will verify both actions by inspecting my commit log. So that I can confidently fix mistakes in any situation, knowing when to rewrite local history and when to add a safe undo commit for shared branches.

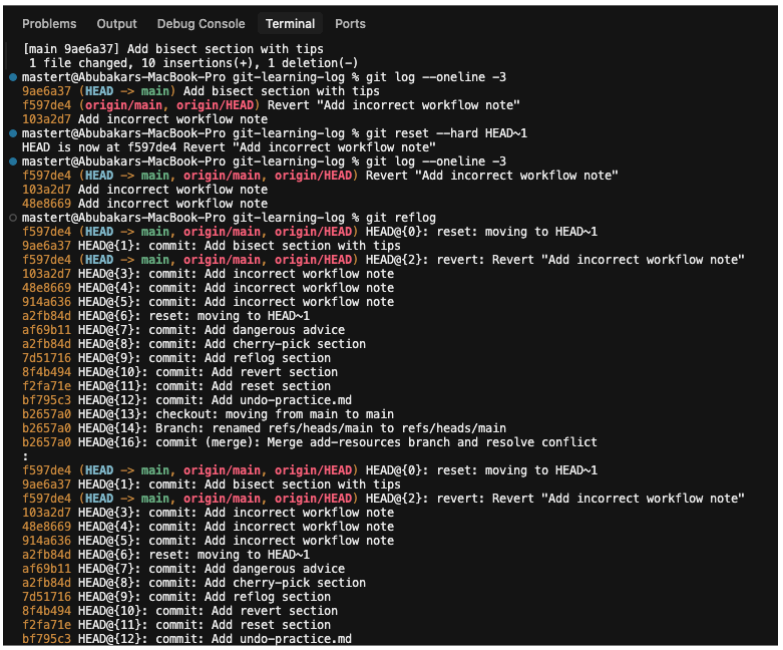
Reset vs. revert: choosing the right tool

I would use git reset when the bad commit is still local and hasn't been pushed to a shared branch. I use it to completely remove the commit from history, which keeps my timeline clean. I would use git revert when the commit has already been pushed to GitHub or shared with others. Revert creates a new commit that undoes the changes without deleting the original commit. I use revert because it preserves history and avoids disrupting collaborators who might have pulled the bad commit. Reset rewrites history safely only locally, while revert adds a safe undo on public branches.

Recovering Lost Commits with Git Reflog

Simulating and recovering from data loss

In this step, I'm simulating a disaster by using git reset --hard to delete a commit and lose work. Then I am using git reflog to find the SHA of that lost commit. I am creating a new branch from that SHA to recover it. Finally, I am merging it back into main. So that I can prove that Git's reflog is a reliable safety net for recovering work after accidental hard resets, giving me confidence to use reset without fear of permanent data loss.



How reflog sees what git log cannot

Git reflog can find the lost commit because it records every single movement of HEAD in my local repository, not just the current commit chain. While git log only shows commits that are reachable from my current branch tip, the reflog keeps a detailed history of all actions like resets, checkouts, and commits. Even after a hard reset removes a commit from the branch history, that commit's SHA and metadata remain in Git's object database and the reflog points to it. So I can always find and recover any commit I made locally within the last 90 days.

learning-log.md undo-practice.md

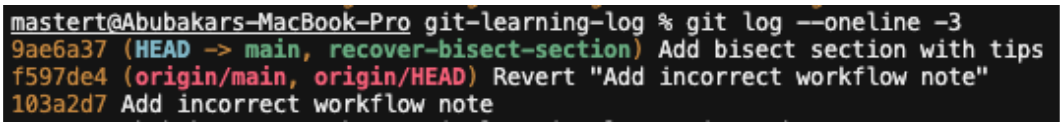
undo-practice.md > # Git Toolkit > ## Bisect

```
# Git Toolkit
## Reset

- git reset --soft HEAD~1: undo last commit, keep changes staged
- git reset --mixed HEAD~1: undo last commit, keep changes in working directory (default)
- git reset --hard HEAD~1: undo last commit and discard all changes (dangerous!)
- Only use reset on commits that haven't been pushed


## Revert

- git revert HEAD: create a new commit that undoes the last commit
- git revert is safe for shared/pushed branches because it doesn't rewrite history
- The original commit stays in the log, plus a new "undo" commit is added
- WRONG: always rebase shared branches to keep history clean

## Reflog

- git reflog: shows everywhere HEAD has pointed (commits, resets, checkouts)
- Reflog entries last about 90 days before being garbage collected
- To recover: find the SHA in reflog, then git branch <name> <SHA>


## Cherry-pick

- git cherry-pick <SHA>: apply a specific commit to the current branch
- Creates a new commit with the same changes but a different SHA
- Use for hotfixes: fix on feature branch, cherry-pick to main


## Bisect

- git bisect start: begin a binary search session
- git bisect bad: mark current commit as containing the bug
- git bisect good <ref>: mark a known-good commit
- Git checks out middle commits; you test and mark good/bad
- git bisect reset: end the session and return to original HEAD
```

Cherry-Picking a Hotfix Across Branches

Setting up the cherry-pick scenario

In this step, I'm setting up a feature branch that contains both unfinished feature work and a separate bug fix commit. I am then switching to main and using git cherry-pick to apply only the fix commit onto main without bringing over any of the incomplete feature changes. I am verifying that main now has the fix but not the unfinished work. So that I can safely deliver urgent production fixes while keeping my feature development separate and incomplete, just like in a real team workflow.



When cherry-pick beats a full merge

I would use cherry-pick instead of merge because it lets me apply only the specific bug fix commit to main without bringing over the unfinished feature work. Merging the entire branch would pull in all three commits, including the incomplete collaboration and advanced tips sections that are not ready for production. Cherry-pick gives me surgical control to select just the critical fix, keeping main stable and deployable. This is essential for hotfixes where speed is important and I cannot wait for the whole feature to be finished and tested.

Hunting Down a Bug with Git Bisect

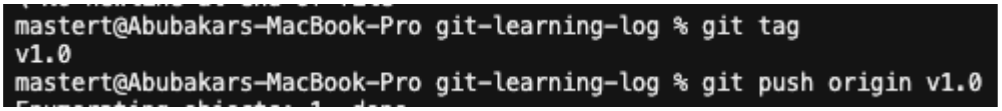
Binary search through commit history

In this step, I'm using git bisect to run a binary search through my commit history and find the exact commit that introduced a hidden bug. I am building a practice branch with several commits, one of which contains an intentional error. I am starting the bisect session, marking known good and bad commits, and testing each midpoint until I identify the culprit. So that I can quickly debug large codebases without manually checking every commit, saving time and reducing frustration in real projects.

Identifying the first bad commit

Git bisect identified the commit with message "Add logging config". This was hard to find manually because the commit message gave no hint about the bug. It said it was adding logging configuration, but it also quietly changed the server port from 3000 to 300. In a real project with dozens or hundreds of commits, I would never think to check a logging commit for a port change. The bug was buried inside a seemingly unrelated change, making manual inspection nearly impossible. Bisect saved me from checking every commit one by one.

Tagging a Release for CI/CD Readiness



Annotated vs. lightweight tags

In this project extension, I learned that annotated tags are for official releases because they store the tagger's name, date, and a message, providing full metadata about the release. I would use these for production deployments and versioned releases like v1.0. Lightweight tags are for private or temporary labels, like marking a checkpoint for my own reference, since they are just simple pointers without extra information. I would use lightweight tags for quick personal notes or testing, while annotated tags are the standard for team workflows and CI/CD triggers.

Reflections and Wrap-Up

Key tools and concepts from this project

The key tools I used include git reset for undoing local commits, git revert for safely undoing pushed changes, git reflog for recovering lost commits, git cherry-pick for applying specific commits, and git bisect for finding bugs through binary search. I also used git tag for marking releases. Key concepts I learnt include the difference between rewriting history locally versus adding history safely on shared branches, the importance of the reflog as a safety net, and how to surgically apply hotfixes without merging unfinished work. I also understood how bisect saves time by automating commit testing.

Time and challenges

This project took me approximately 50 minutes to complete. The most challenging part was using git bisect correctly because I had to carefully test each middle commit and mark it as good or bad without making a mistake. I also found recovering a lost commit with reflog a bit tricky at first, but once I understood how to find the SHA and create a new branch, it became clear. Overall, the hands-on practice with reset and revert really helped me understand when to use each command safely in real projects.

Looking ahead

I did this project today to learn how to safely undo mistakes, recover lost work, and debug history using Git's core recovery tools. I now feel confident using reset, revert, reflog, cherry-pick, and bisect in real projects. Another skill I want to learn is how to resolve complex merge conflicts and use interactive rebase to clean up commit history before merging. I also want to understand Git hooks and automation for better workflow control. This project gave me a solid foundation, and I am excited to keep building on these skills in future challenges.